



IEEE 754 Decimals for C++

The Boost.Decimal Library

MATT BORLAND



20
26



Start with a Quiz

What is the output of this program?

```
#include <iomanip>
#include <iostream>

int main() {
    std::cout << std::setprecision(17) << 0.1 + 0.2;
}
```

0.300000000000000000000004

Why is this?

Where the extra came from

The bitwise representation of float `f = 0.1f`;



Five does not divide two

A fraction terminates in base b only if every prime factor of its denominator divides b .¹

$$\frac{1}{10} \quad 10 = 2 \times 5 \quad 5 \nmid 2$$

No number of bits fixes this. The problem is not precision, but **representability**.

¹ Hardy & Wright, [An Introduction to the Theory of Numbers](#), §9.2–9.3.

The solution is not "more precision"

These numerical types we are about to discuss are useful when these errors are unacceptable.

- Canonical example and many users are in the financial sector
- Human-entered data and databases
- Legal and regulatory compliance

A brief history

- **IEEE 754-2008** specifies decimal floating point — two encodings, three interchange formats
- **ISO/IEC TR 24733** (2011) sketches a C++ binding. Never adopted into the standard.
- **C23** adds `_Decimal32` `_Decimal64` `_Decimal128` — as an optional feature
- **GCC** ships the types on some targets, with no accompanying standard library
- **IBM's libdfp** fills the library gap
- **Intel** ships a library with its own types

Boost.Decimal

- Header-only, **no dependencies**, C++14
- Six types: `decimal32_t` `decimal64_t` `decimal128_t`
and `decimal_fast32_t` `decimal_fast64_t` `decimal_fast128_t`
- `constexpr` throughout
- Its own `<cmath>` `<charconv>` `<format>`, etc.
- Boost.Math integration
- CUDA support for the types and much of the math
- Tested on x86_64, ARM64, s390x, 32-bit; emulated PPC64LE and Cortex-M

There is a JOSS paper — full citation at the end.

Part 1 of 5

1. Inside the bits
2. **The Types**
3. The Standard Library
4. Performance
5. When to reach for it

A Binary Floating Point Refresher

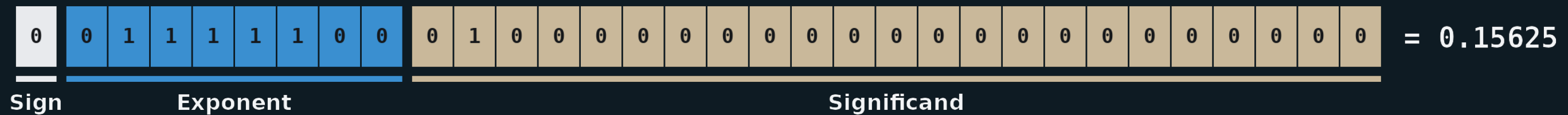


float: 1 + 8 + 23 = 32.

$$(-1)^0 \times 1.01_2 \times 2^{124-127} = 1.25 \times 2^{-3} = 0.15625$$

Same value with one additional field.

float



decimal32_t



$$15625 \times 10^{-5} = 0.15625$$

Why the extra field?

`decimal32_t` holds 7 decimal digits -> max significand 9,999,999

$2^{23} = 8,388,608 < 9,999,999$

$2^{24} = 16,777,216$

You need 24 bits, but sign and exponent leave you with only 23.

Four cases

steer	layout				sig. bits
00	s		eeeeeeee	ttttttttttttttttttttttttttttttt	23
01	s		eeeeeeee	ttttttttttttttttttttttttttttttt	23
10	s		eeeeeeee	ttttttttttttttttttttttttttttttt	23
11	s	11	eeeeeeee	[100] + ttttttttttttttttttttttttttttt	24

Watch the fields move for adjacent integers

```
decimal32_t{8388607} steering != 11
```



```
decimal32_t{8388608} steering == 11
```



0.1, both ways

float



decimal32_t



BID and DPD

IEEE 754 specifies two encodings:

- **BID**: Binary Integer Significand. Intended for software implementations.
- **DPD**: Densely Packed Decimal. Intended for hardware implementations.

Boost.Decimal uses **BID**, and can convert either way:

```
constexpr std::uint32_t to_bid_d32(decimal32_t val) noexcept;  
constexpr decimal32_t from_bid_d32(std::uint32_t bits) noexcept;  
constexpr std::uint32_t to_dpd_d32(decimal32_t val) noexcept;  
constexpr decimal32_t from_dpd_d32(std::uint32_t bits) noexcept;
```


BID vs DPD for our adjacent integers

	BID	DPD
<code>decimal32_t{8388607}</code>	<code>0x32FFFFFFFF</code>	<code>0x6A573B07</code>
<code>decimal32_t{8388608}</code>	<code>0x6CA00000</code>	<code>0x6A573B08</code>

BID rebuilds the word.

DPD increments by one.

Now the part with no binary analogue

One value. Seven encodings.

The set of representations a floating-point number maps to is called the floating-point number's cohort.

— IEEE 754-2008

<code>decimal32_t{</code>	<code>3,</code>	<code>2}</code>	<code>-></code>	<code>0x33800003</code>
<code>decimal32_t{</code>	<code>30,</code>	<code>1}</code>	<code>-></code>	<code>0x3300001E</code>
<code>decimal32_t{</code>	<code>300,</code>	<code>0}</code>	<code>-></code>	<code>0x3280012C</code>
<code>decimal32_t{</code>	<code>3000000,</code>	<code>-4}</code>	<code>-></code>	<code>0x30ADC6C0</code>

- All compare equal with operator==
- All **hash equal**
- None are bitwise equal

Normalizing

A value is **normalized** when the cohort effect is removed by appending zeros to the significand and reducing the exponent until it carries the type's full precision:

$$3 \times 10^2 \longrightarrow 3000000 \times 10^{-4}$$

Every arithmetic operation and every comparison has to handle the effects of cohorts.

This is among the most expensive parts of these operations.

Part 2 of 5

1. Inside the bits
2. **The Types**
3. The Standard Library
4. Performance
5. When to reach for it

One Include

```
#include <boost/decimal.hpp>
```

Header-only. No dependencies. C++14.

```
import boost.decimal; // C++20 module
```

- Available individually through vcpkg and conan, or with Boost in most package managers
- Build with b2 or CMake

Six types

Three IEEE 754 compliant types:

	bytes	digits ¹⁰	exponent	range
decimal32_t	4	7	-95 ..	+96
decimal64_t	8	16	-383 ..	+384
decimal128_t	16	34	-6143 ..	+6144

Three performance-focused types:

decimal_fast32_t	8	7	-95 ..	+96
decimal_fast64_t	16	16	-383 ..	+384
decimal_fast128_t	32	34	-6143 ..	+6144

The performance costs exactly double the size.

What the extra bytes buy

```
class decimal32_t final {  
    std::uint32_t bits_;           // IEEE 754 BID  
};
```

```
class decimal_fast32_t final {  
    std::uint32_t significand_;    // stored decoded and normalized  
    std::uint8_t  exponent_;  
    bool          sign_;  
};
```


Which one to use?

Does the bit pattern matter?

- **Yes:** a file, a socket, a column, another language, another vendor
→ `decimal32_t` `decimal64_t` `decimal128_t`
- **No:** computed, compared, and discarded
→ `decimal_fast32_t` `decimal_fast64_t` `decimal_fast128_t`

Construction

```
decimal32_t a {1, 1};           // 1e1
decimal32_t b {-2, -1};         // -0.2
decimal32_t c {2U, -1, construction_sign::negative}; // -0.2
decimal32_t d {"4.3e-02"};      // string or string_view
auto       e {"3.14159265358979"_DF}; // literal, rounds to fit
```

It promotes like you expect

<code>decimal32_t{}</code>	<code>+</code>	<code>decimal64_t{}</code>	<code>-></code>	<code>decimal64_t</code>
<code>decimal64_t{}</code>	<code>*</code>	<code>2</code>	<code>-></code>	<code>decimal64_t</code>
<code>decimal64_t{}</code>	<code>+</code>	<code>decimal_fast32_t{}</code>	<code>-></code>	<code>decimal64_t</code>
<code>decimal_fast128_t{}</code>	<code>-</code>	<code>decimal128_t{}</code>	<code>-></code>	<code>decimal_fast128_t</code>

Wider precision wins. On a tie, **fast wins**.

What it will not do quietly

```
decimal64_t d {3.14};           // explicit only, and discouraged  
int n = d;                       // ill-formed
```

Part 3 of 5

1. Inside the bits
2. The Types
3. **The Standard Library**
4. Performance
5. When to reach for it

What comes in the box

<code><cmath></code>	over a hundred functions, all constexpr
<code><charconv></code>	<code>to_chars</code> / <code>from_chars</code> , plus two decimal-only formats
<code><format></code>	and <code>{fmt}</code>
<code><cstdlib></code>	<code>strtod32</code> / <code>strtod64</code> / <code>strtod128</code>
<code><cfenv></code>	rounding modes
<code><cfloat></code>	evaluation modes
<code><functional></code>	<code>std::hash</code> , all six types
<code><limits></code>	<code>numeric_limits</code> , all six types
<code><string></code>	<code>to_string</code> , <code>stod32</code> ...
<code><iostream></code>	<code>operator<<</code> and <code>operator>></code>
<code><numbers></code>	<code>pi</code> , <code>e</code> , <code>ln2</code> , ... per type

Everything lives in `boost::decimal`, except literals, which live in `boost::decimal::literals`.

Generic code: the `std::swap` two-step

```
template <typename T>
void sin_identity(T val)
{
    using std::sin;
    using boost::decimal::sin;

    sin(val);           // unqualified. float, double, decimal128_t – all fine.
}
```

All of it is constexpr

```
constexpr decimal64_t two {2};  
constexpr decimal64_t root {sqrt(two)};  
  
static_assert(root == "1.414213562373095"_DD, "");  
  
std::sqrt is not constexpr until C++26.
```


Two ways to decompose a value

Three spellings of 300, taken apart two ways:

decimal32_t{300}	decompose	->	sig 300,	exp 0
	frexp10	->	sig 3000000,	exp -4
decimal32_t{3, 2}	decompose	->	sig 3,	exp 2
	frexp10	->	sig 3000000,	exp -4
decimal_fast32_t{3, 2}	decompose	->	sig 3000000,	exp -4
	frexp10	->	sig 3000000,	exp -4

- decompose retains the cohort
- frexp10 normalizes it away
- The fast types return the same for both since they store normalized

A deeper look into `<charconv>`

chars_format options

```
enum class chars_format : unsigned
{
    scientific,
    fixed,
    hex,
    cohort_preserving_scientific, // not in <charconv>
    cohort_preserving_fixed,     // not in <charconv>
    general = fixed | scientific
};
```

The round trip

<charconv> using
chars_format::cohort_preserving_scientific

"3e+02"	->	0x33800003	->	"3e+02"
"3.0e+02"	->	0x3300001E	->	"3.0e+02"
"3.00e+02"	->	0x3280012C	->	"3.00e+02"
"3.000e+02"	->	0x32000BB8	->	"3.000e+02"
"3.0000e+02"	->	0x31807530	->	"3.0000e+02"
"3.00000e+02"	->	0x310493E0	->	"3.00000e+02"
"3.000000e+02"	->	0x30ADC6C0	->	"3.000000e+02"

Sizing buffers

```
template <typename DecimalType, int Precision = std::numeric_limits<DecimalType>::max_digits10>
class formatting_limits
{
public:
    static constexpr std::size_t scientific_format_max_chars;
    static constexpr std::size_t fixed_format_max_chars;
    static constexpr std::size_t hex_format_max_chars;
    static constexpr std::size_t cohort_preserving_scientific_max_chars;
    static constexpr std::size_t cohort_preserving_fixed_max_chars;
    static constexpr std::size_t general_format_max_chars;
    static constexpr std::size_t max_chars;
};
```

This simplifies correct buffer sizing to:

```
char buf[formatting_limits<decimal64_t>::scientific_format_max_chars];
```

Integration with other libraries

Two ways to format

	needs	standard
<code><fmt/format.h></code>	{fmt} 11 or newer	C++14
<code><format></code>	GCC 13 · Clang 18 · MSVC 19.40	C++20

<code>fmt::format("{:*>+12.2e}", val);</code>	<code>// [***+3.14e+00]</code>
<code>std::format("{:*>+12.2e}", val);</code>	<code>// [***+3.14e+00]</code>

One extra formatting specifier

g	G	general
e	E	scientific
f		fixed
x	X	hex
a	A	cohort preserving scientific

```
fmt::format("{ }", d);           // 300  
fmt::format("{:a}", d);          // 3.00e+02
```

Rounding

Binary Floating Point Rounding

In `<cfenv>` we have the following four macros useable with `std::fesetround()`:

1. `FE_DOWNWARD` – Rounding towards negative infinity
2. `FE_TONEAREST` – Rounding towards nearest representable value
3. `FE_TOWARDZERO`– Rounding towards zero
4. `FE_UPWARD` – Rounding towards positive infinity

The Five Decimal Rounding Modes

All modes are in enum class `rounding_mode` and useable with `boost::decimal::fesetround()`:

1. `fe_dec_downward`
2. `fe_dec_to_nearest` - To nearest, ties to even
3. `fe_dec_to_nearest_from_zero`
4. `fe_dec_toward_zero` - The opposite of 3
5. `fe_dec_upward`

Default is #2, per IEEE 754 4.3.3 and is called "banker's rounding", and is available also as `fe_dec_default`

Example of Rounding Mode

```
#include <boost/decimal.hpp>
#include <iostream>

int main() {
    using namespace boost::decimal::literals;
    using boost::decimal::decimal32_t;

    boost::decimal::fesetround(boost::decimal::rounding_mode::fe_dec_upward); // NOT THREAD-SAFE

    const decimal32_t lhs {"5e+50"_DF};
    const decimal32_t rhs {"4e+40"_DF};
    const decimal32_t sum {lhs + rhs};

    std::cout << "5e50 + 4e40 = " << sum << std::end;
}
```

Output: 5e50 + 4e40 = 5.000001e+50

Compile Time Rounding

Very similar to the run-time enum class, but we now have the following macros:

1. `BOOST_DECIMAL_FE_DEC_DOWNWARD`
2. `BOOST_DECIMAL_FE_DEC_TO_NEAREST`
3. `BOOST_DECIMAL_FE_DEC_TO_NEAREST_FROM_ZERO`
4. `BOOST_DECIMAL_FE_DEC_TOWARD_ZERO`
5. `BOOST_DECIMAL_FE_DEC_UPWARD`

Changing the Compile Time Rounding Mode

```
#define BOOST_DECIMAL_FE_DEC_DOWNWARD    // before ANY decimal header

#include <boost/decimal/decimal32_t.hpp>
#include <boost/decimal/literals.hpp>

using namespace boost::decimal::literals;
using boost::decimal::decimal32_t;

constexpr decimal32_t lhs {"5e+50"_DF};
constexpr decimal32_t rhs {"4e+40"_DF};
constexpr decimal32_t res {lhs - rhs};

static_assert(res == "4.999999e+50"_DF, "Incorrectly rounded result");
```

Boost.Math Integration

Bollinger bands over a year of AAPL closes.

```
#include <boost/math/statistics/univariate_statistics.hpp>
#include <boost/decimal.hpp>

std::vector<decimal64_t> closes;

// Import closing data from a data source of AAPL

const decimal64_t mean    = boost::math::statistics::mean(closes);
const decimal64_t median  = boost::math::statistics::median(closes);
const decimal64_t var     = boost::math::statistics::variance(closes);
const decimal64_t sigma   = boost::decimal::sqrt(var);
```

```
    Mean Closing Price: $207.20
    Standard Deviation: $25.45
Upper Bollinger Band: $258.11
Lower Bollinger Band: $156.30
```


On the GPU

```
#define BOOST_DECIMAL_ENABLE_CUDA
#include <boost/decimal/decimal64_t.hpp>

__global__ void add(const decimal64_t* a, const decimal64_t* b,
                    decimal64_t* out, int n)
{
    const int i {blockDim.x * blockIdx.x + threadIdx.x};
    if (i < n) { out[i] = a[i] + b[i]; }
}
```

Device results compare to the same loop run on the host

Debugging Pretty Printers

`decimal32_t` - cohort preserving

```
> [[ max = {const boost::decimal::decimal32_t} 9.999999e+96
> [[ min = {const boost::decimal::decimal32_t} 1e-95
> [[ short_num = {const boost::decimal::decimal32_t} 3.140e+00
> [[ pos_inf = {const boost::decimal::decimal32_t} INF
> [[ neg_inf = {const boost::decimal::decimal32_t} -INF
> [[ qnan = {const boost::decimal::decimal32_t} QNAN
> [[ snan = {const boost::decimal::decimal32_t} SNAN
> [[ payload_nan = {const boost::decimal::decimal32_t} QNAN(7)
```

`decimal_fast32_t` - always normalized

```
> [[ max = {const boost::decimal::decimal_fast32_t} 9.999999e+96
> [[ min = {const boost::decimal::decimal_fast32_t} 1.000000e-95
> [[ short_num = {const boost::decimal::decimal_fast32_t} 3.140000e+00
> [[ pos_inf = {const boost::decimal::decimal_fast32_t} INF
> [[ neg_inf = {const boost::decimal::decimal_fast32_t} -INF
> [[ qnan = {const boost::decimal::decimal_fast32_t} QNAN
> [[ snan = {const boost::decimal::decimal_fast32_t} SNAN
> [[ payload_nan = {const boost::decimal::decimal_fast32_t} QNAN(7)
```


Part 4 of 5

1. Inside the bits
2. The Types
3. The Standard Library
4. **Performance**
5. When to reach for it

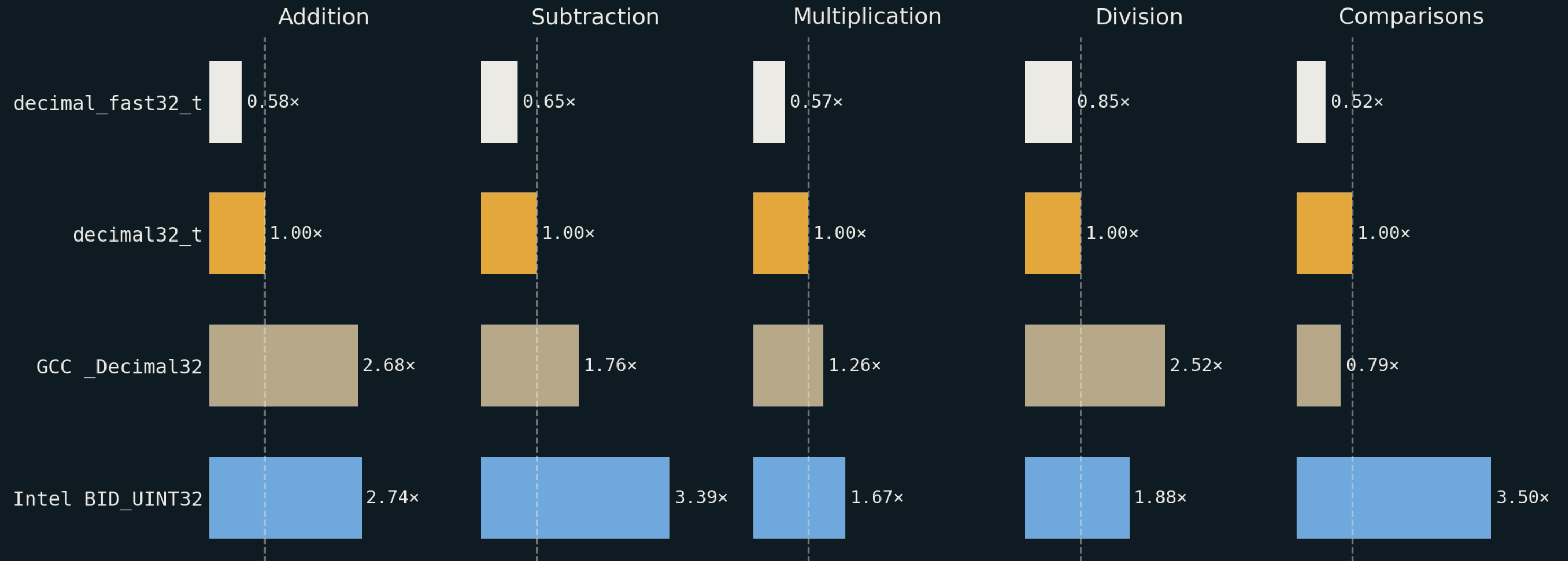
Benchmarks

Complete Benchmarks are available for the following systems:

1. x86_64 Linux
2. x86_32 Linux
3. x86_64 Windows
4. ARM64 Windows
5. ARM64 MacOS

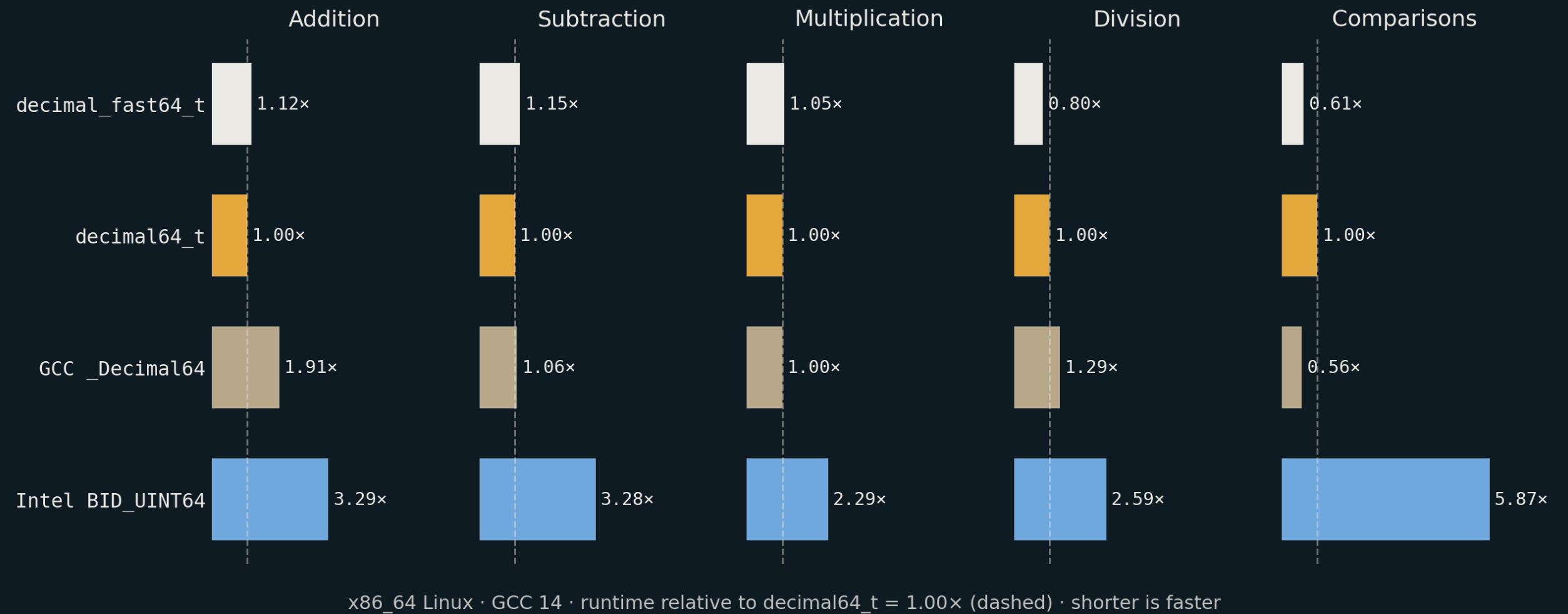
`GCC _DecimalXX` and `Intel BID_UINTXX` are included where available for completeness

32-bit Types



x86_64 Linux · GCC 14 · runtime relative to decimal32_t = 1.00x (dashed) · shorter is faster

64-bit Types



One honest loss: `GCC _Decimal64` compares faster than we do

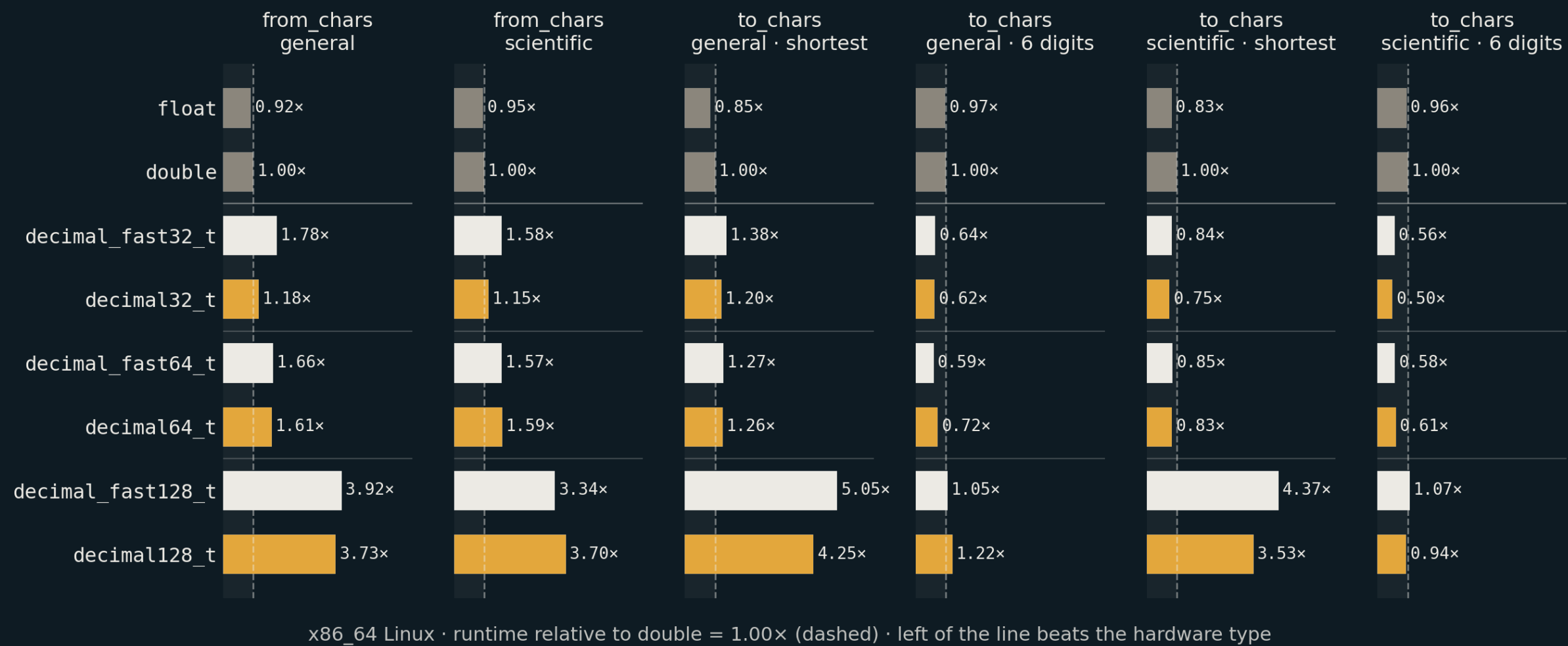
128-bit Types



x86_64 Linux · GCC 14 · runtime relative to decimal128_t = 1.00x (dashed) · shorter is faster

The closest race of the three widths

Where decimal beats the hardware



— <charconv> is software for every type so the hardware advantage disappears

Part 5 of 5

1. Inside the bits
2. The Types
3. The Standard Library
4. Performance
5. **When to reach for it**

Use Cases Revisited

At the top of the presentation we suggested:

- Canonical example and many users are in the financial sector
- Human-entered data and databases
- Legal and regulatory compliance

Current Clients Applications

- A trading firm, because their home-brew fixed point could not represent the smallest divisible unit of a Bitcoin (A Satoshi = $1/100,000,000$ of a Bitcoin)
- Several quant firms, who use `<charconv>` extensively
- A database firm, who needed `decimal128_t` to be run on ARM
- Boost.MySQL for the DECIMAL data type (in conjunction with Boost.Multiprecision)

Takeaways

- Decimal Floating Point types can provide a new way to handle data and computations, but there is a performance cost
- Boost.Decimal is portable, performant, and available everywhere.

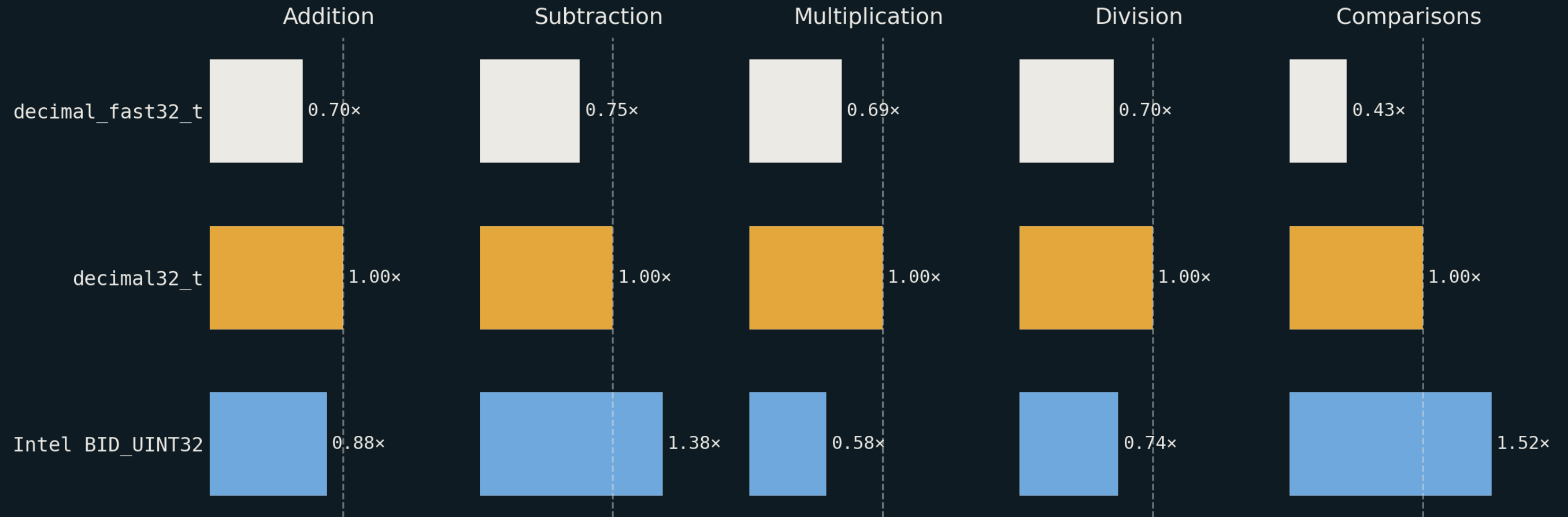
Questions and Citation

Borland and Kormanyos, (2026). Boost.Decimal: A C++14 Library for Decimal Floating Point Arithmetic. Journal of Open Source Software, 11(124), 10345, <https://doi.org/10.21105/joss.10345>

There is also a CITATION.cff file in the repo

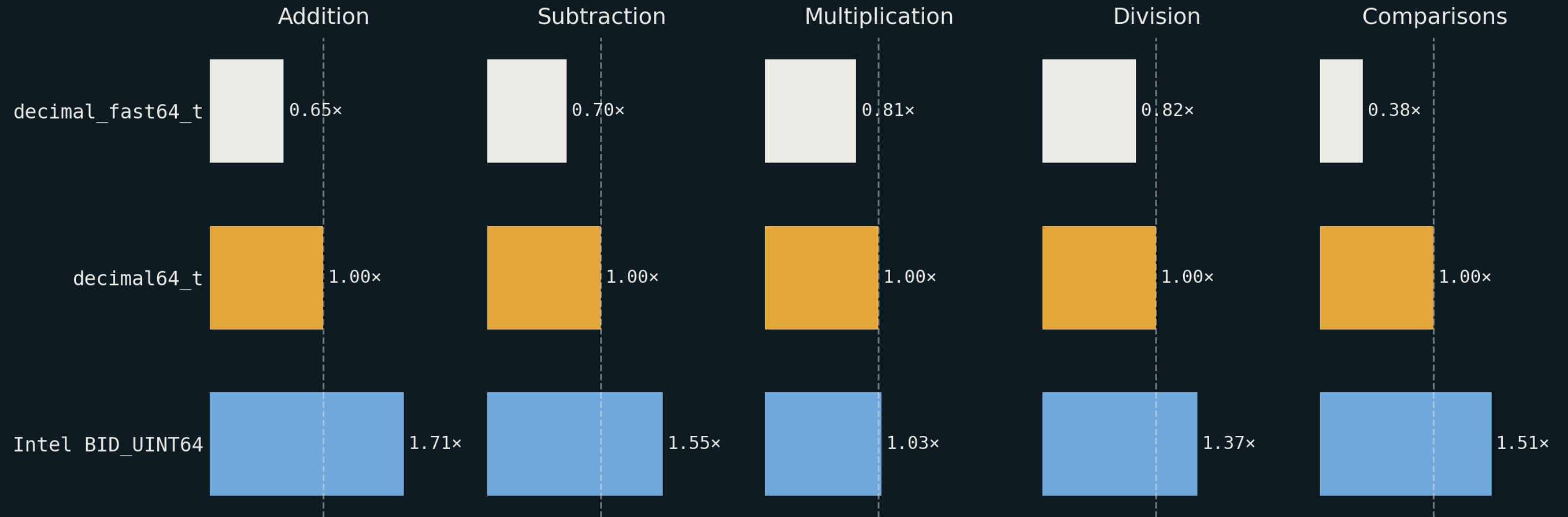
Appendix

x86_64 Linux · Intel oneAPI — 32 bits



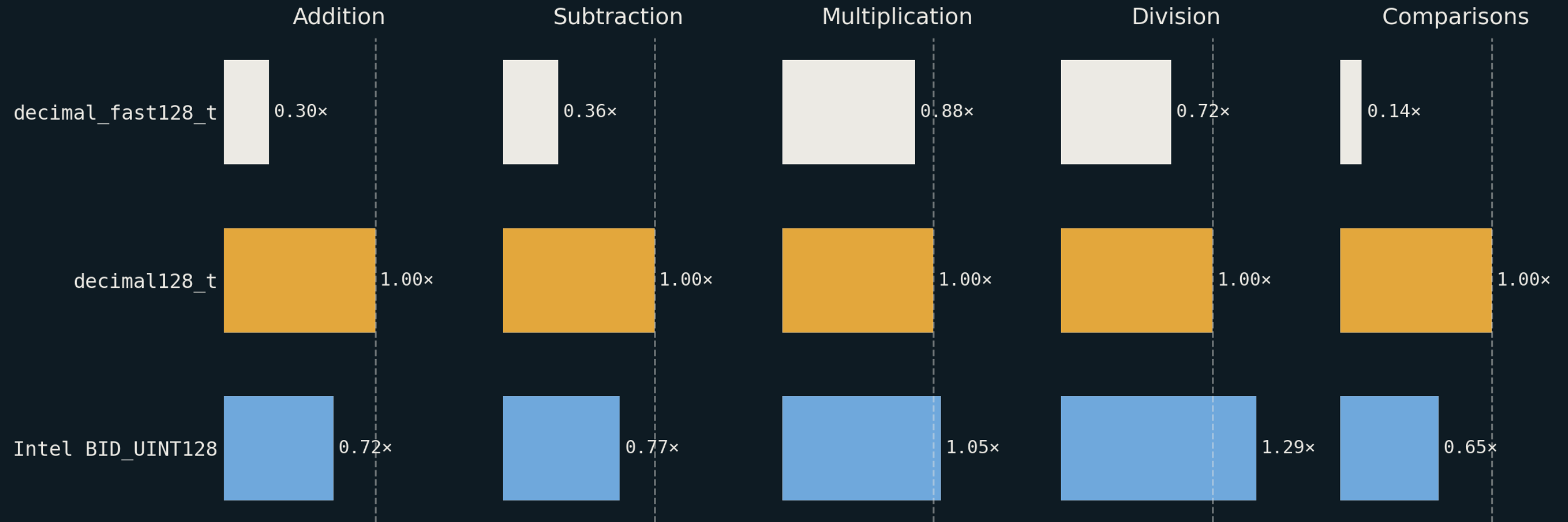
x86_64 Linux · Intel oneAPI (icpx / icx) · runtime relative to `decimal32_t` = 1.00x (dashed) · shorter is faster

x86_64 Linux · Intel oneAPI — 64 bits



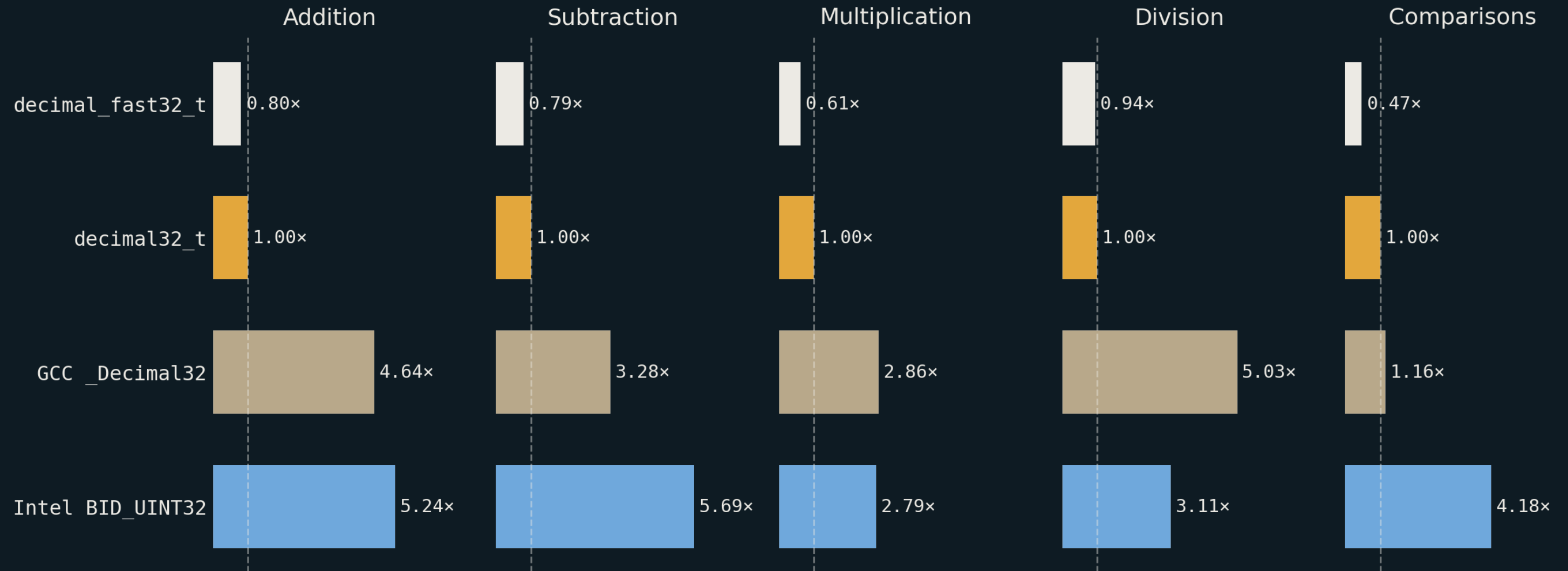
x86_64 Linux · Intel oneAPI (icpx / icx) · runtime relative to decimal64_t = 1.00x (dashed) · shorter is faster

x86_64 Linux · Intel oneAPI — 128 bits



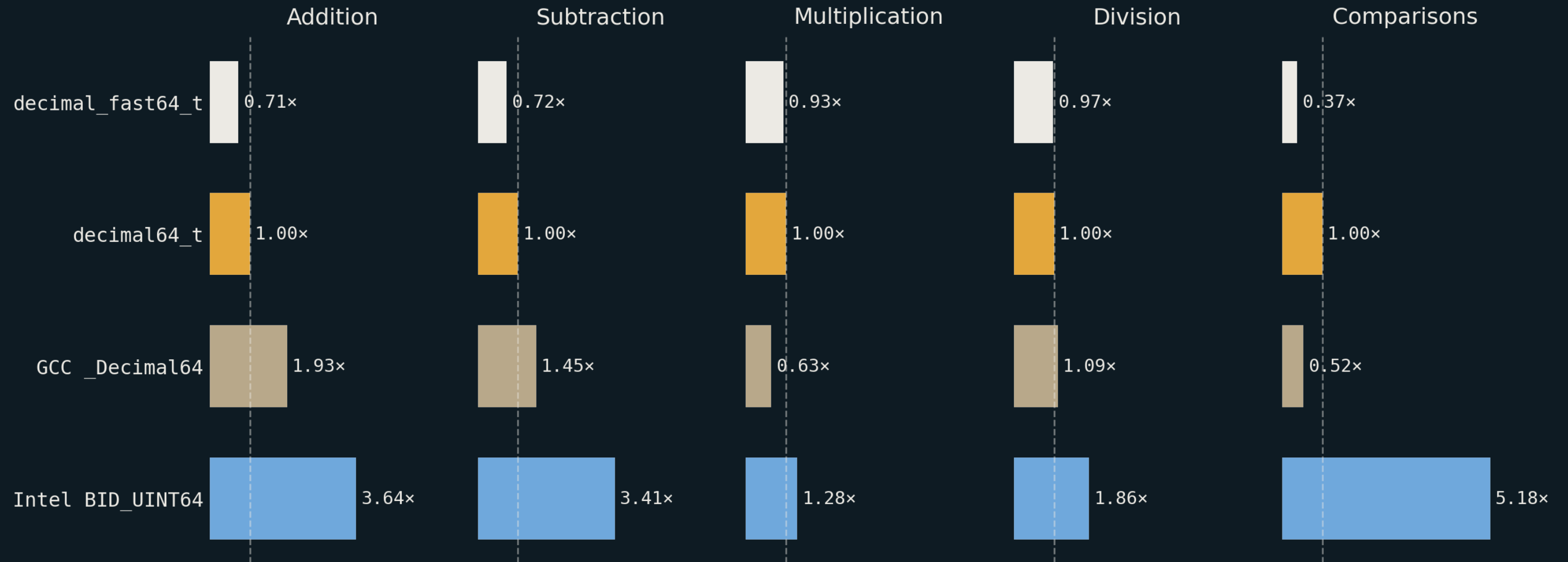
x86_64 Linux · Intel oneAPI (icpx / icx) · runtime relative to `decimal128_t` = 1.00x (dashed) · shorter is faster

x86_32 Linux · GCC 14 (-m32) — 32 bits



x86_32 Linux · GCC 14 · -m32 · runtime relative to `decimal32_t` = 1.00x (dashed) · shorter is faster

x86_32 Linux · GCC 14 (-m32) — 64 bits



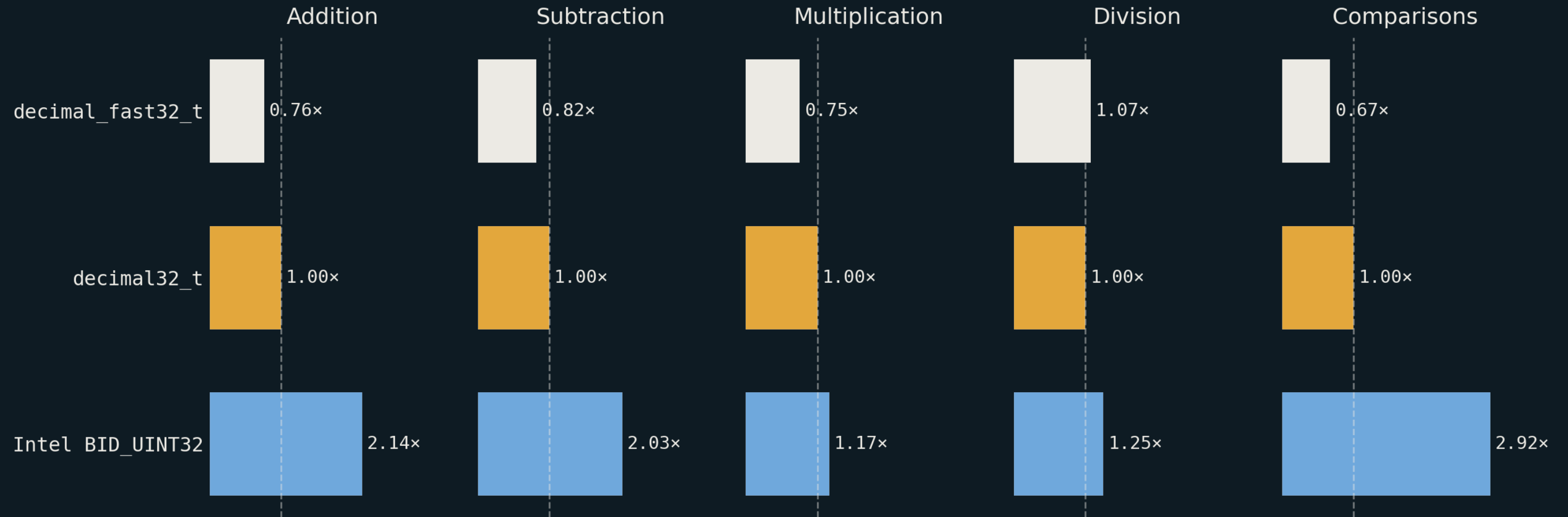
x86_32 Linux · GCC 14 · -m32 · runtime relative to `decimal64_t` = 1.00x (dashed) · shorter is faster

x86_32 Linux · GCC 14 (-m32) — 128 bits



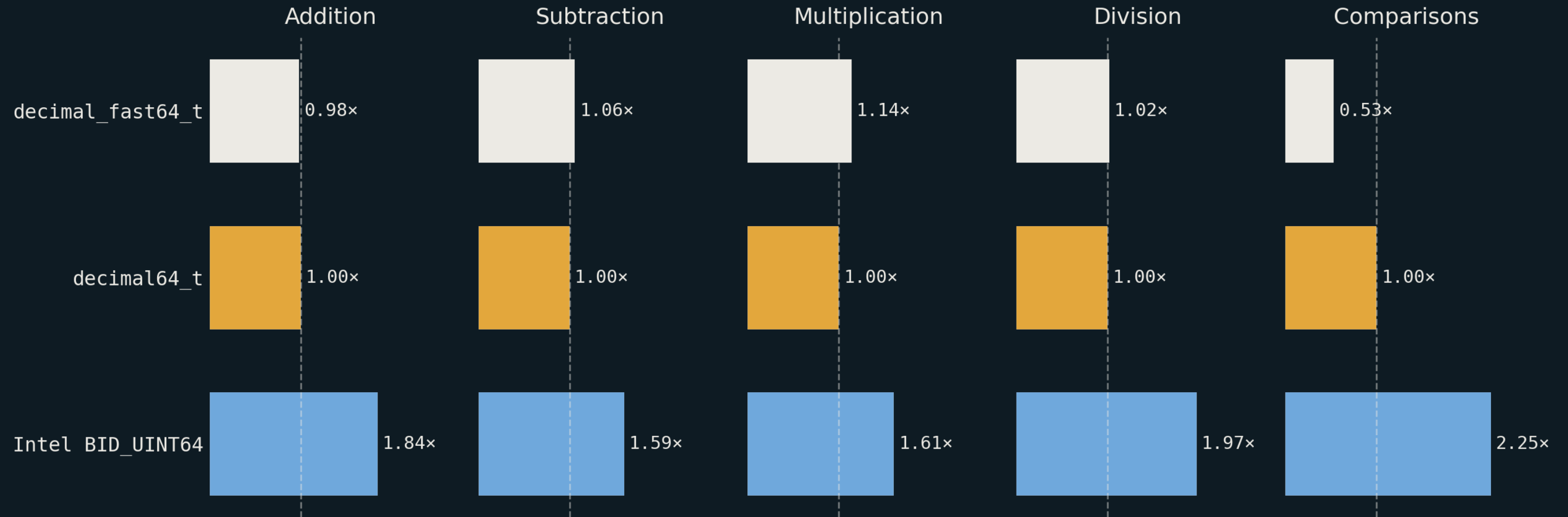
x86_32 Linux · GCC 14 · -m32 · runtime relative to decimal128_t = 1.00× (dashed) · shorter is faster

x86_64 Windows · MSVC — 32 bits



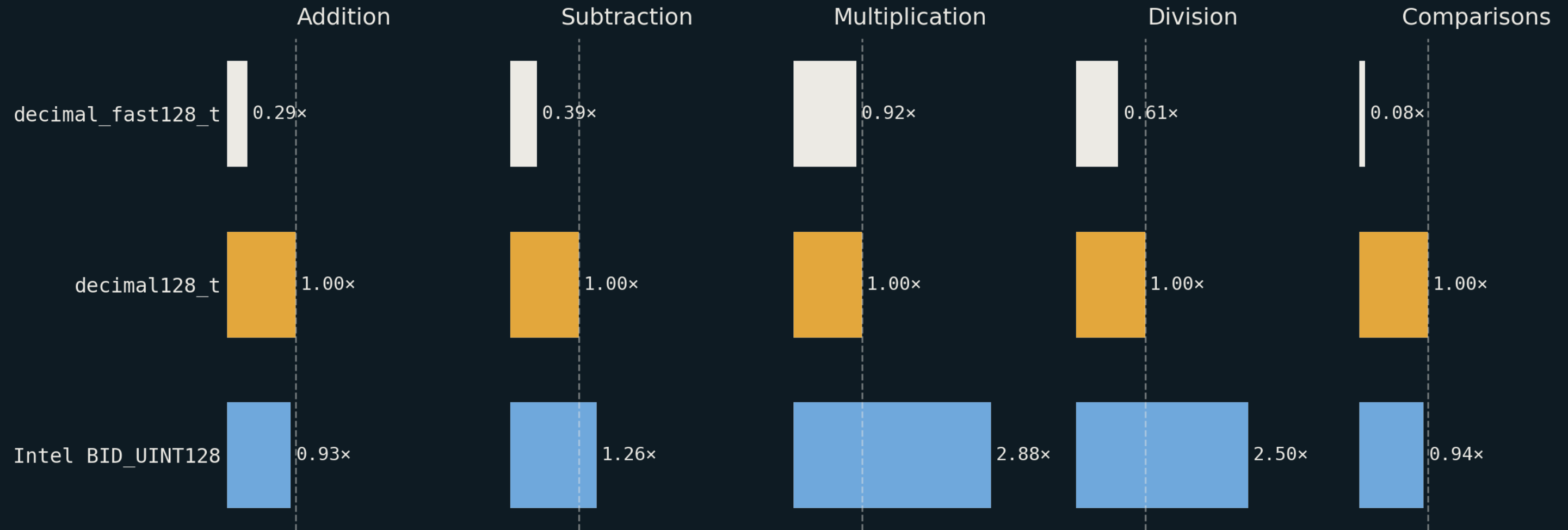
x86_64 Windows · MSVC · runtime relative to `decimal32_t` = 1.00x (dashed) · shorter is faster

x86_64 Windows · MSVC — 64 bits



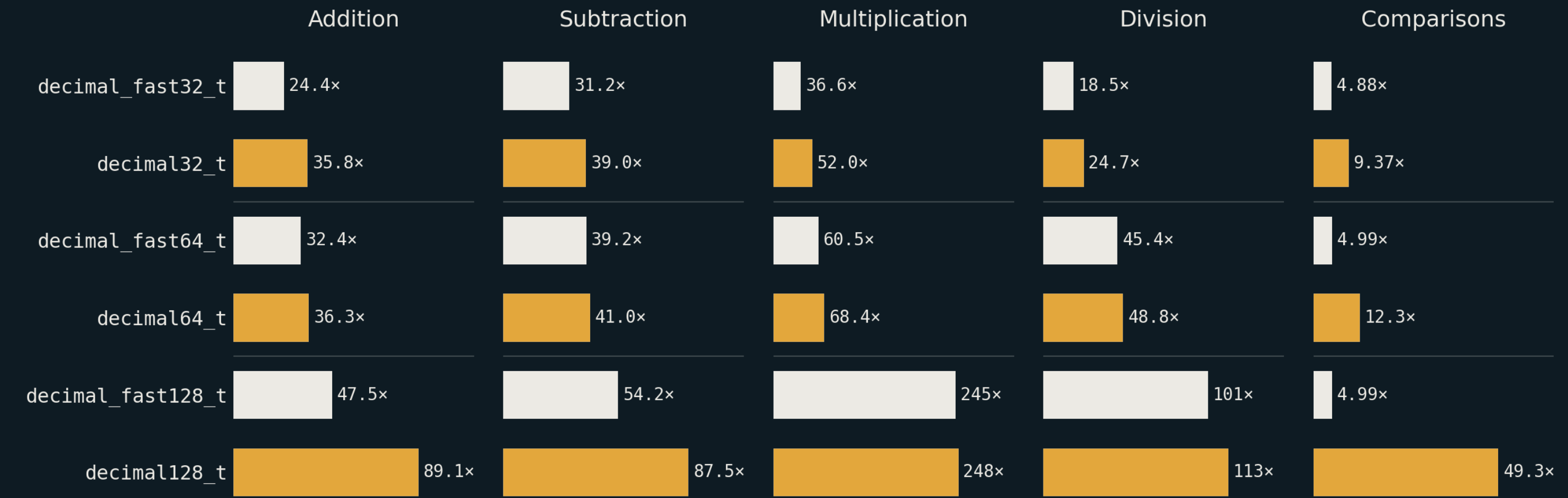
x86_64 Windows · MSVC · runtime relative to `decimal64_t` = 1.00x (dashed) · shorter is faster

x86_64 Windows · MSVC — 128 bits



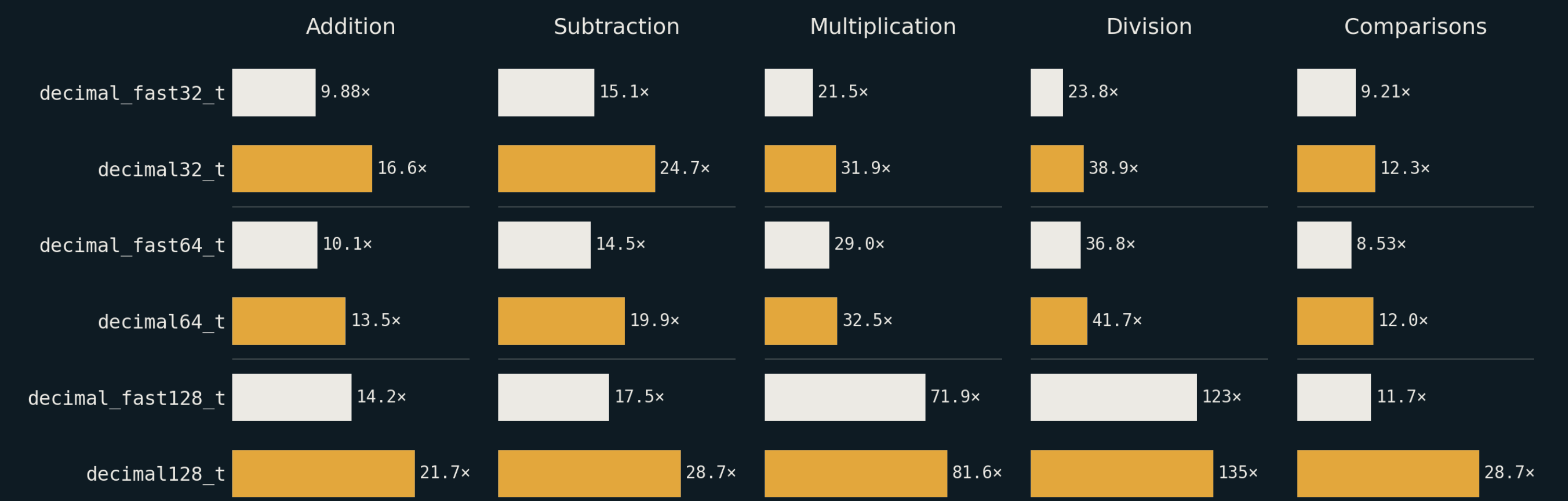
x86_64 Windows · MSVC · runtime relative to `decimal128_t` = 1.00x (dashed) · shorter is faster

ARM64 Windows · MSVC



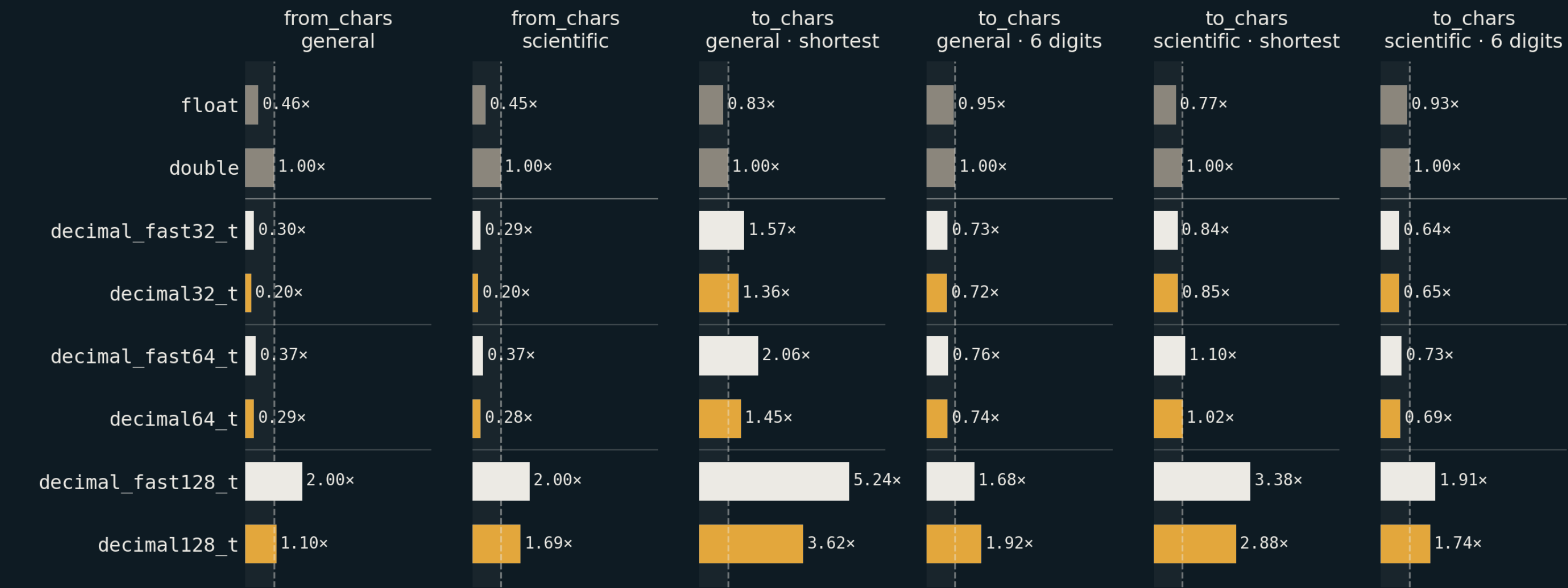
ARM64 Windows · MSVC · runtime relative to hardware double = 1.00× · shorter is faster · panels scaled independently

ARM64 macOS · M4 Max



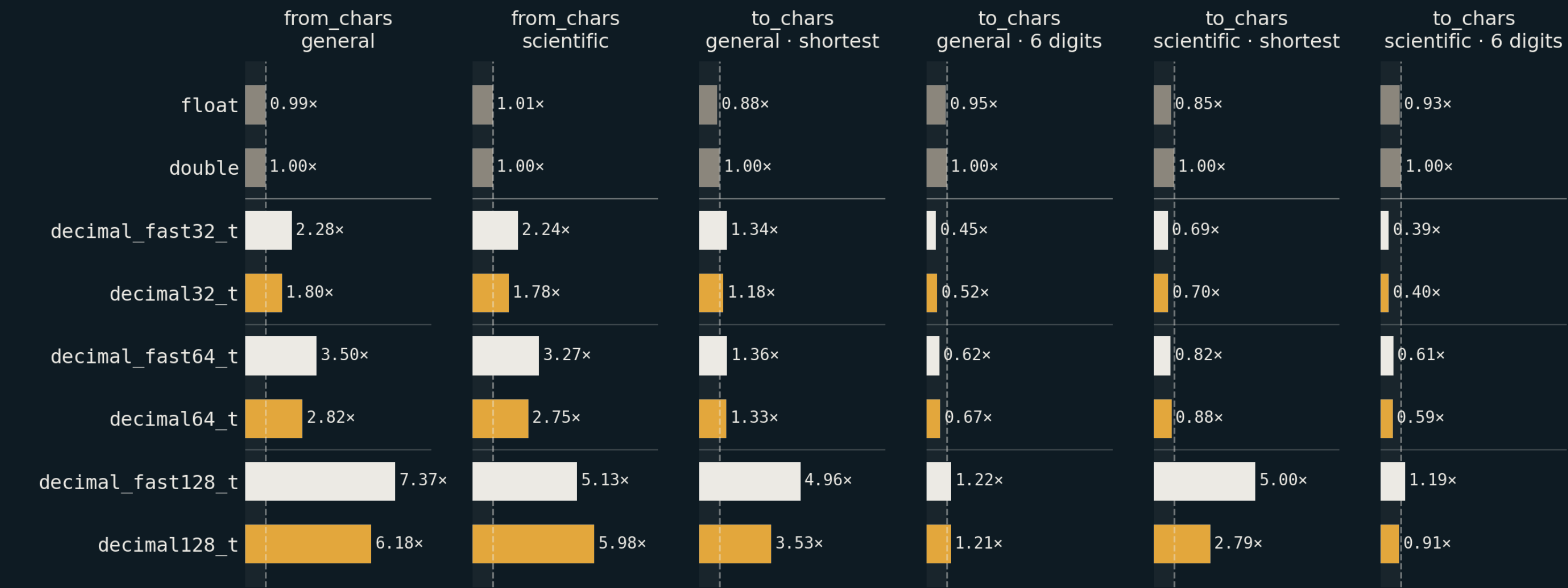
ARM64 macOS · M4 Max · Clang 22 · runtime relative to hardware double = 1.00×

<charconv> — x86_64 Windows · MSVC



x86_64 Windows · MSVC · runtime relative to double = 1.00x (dashed) · left of the line beats the hardware type

<charconv> — ARM64 macOS



ARM64 macOS · M4 Max · Clang 22 · runtime relative to double = 1.00x (dashed) · left of the line beats the hardware type